Document Version: 1.0
Runner Version: 1.0.0
Test Version: 1.0.0

This guide explains how to use the LiteTest Runner framework and Runner API covering concepts, workflow, execution model, examples, and practical,usage patterns. It complements the LiteTest Runner Reference, which documents the Runner API in detail.

Preface

This document is intended for LiteTest contributors who need guidance on enhancing and maintaining LiteTest.

For a list of other LiteTest documents and the LiteTest repository layout, see the LiteTest Documentation Guide.

For a glossary of terms, see the LiteTest Glossary Reference.

Document Version History

Document	Runner	Test	Date	Comment	Author/Editor
1.0	1.0.0	1.0.0	2026-06-11	Initial version.	Paul Sinclair

- The **Document** column records the document's version with the format `M.u` (Major, update).
- The **Runner** column records the LiteTest Runner API version current at the time this document version was published and is defined by its `LT_RUNNER_VERSION` macro.
- The **Test** column records the LiteTest Test API version current at the time this document version was published and is defined by its `LT_TEST_VERSION` macro.
- Both Runner and Test use the version format `"M.m.p"` (Major, minor, patch).
- `M` is the same for the Document, Runner, and Test versions.

The document's update version tracks released updates to this document and does not correspond to a minor or patch version. `u` increments when document changes are published in a release without a change to `M`, and it resets to `0` when `M` is incremented.

Table of Contents

1. Introduction 18

1. Introduction

LiteTest is built around three ideas:

1. **Test expressions**:C expressions wrapped with `LT_TEST`.
2. **Test group functions**:Functions that contain related tests.
3. **The orchestrator**: A `main` function that runs test groups and writes the report.

A typical test executable includes:

- An orchestrator (`main`)
- One or more test group functions
- `litetest_runner.h` and `litetest_runner.c`
- Project headers and sources under test

2. Execution Model

LiteTest runs test expressions and test group functions under a configurable isolation model:

- **Same-thread execution** (default)
- **Thread-isolated execution**
- **Process-isolated execution**

The orchestrator and each test group function specify a `maxparallel` value that controls how many tests or groups may run concurrently.

Faults such as `SIGSEGV`, `SIGBUS`, `SIGILL`, and `SIGFPE` are detected and reported without aborting the test run.

3. Test Group Functions

A test group function:

- Is declared with `LT_DECLARE_GROUP`
- Begins with `LT_INIT_GROUP`
- Contains `LT_TEST` and/or `LT_GROUP` calls
- Ends with `LT_RETURN`

Example:

```
static LT_DECLARE_GROUP(test_math)
{
  LT_INIT_GROUP(test_math, 1);

  LT_TEST(1 + 1 == 2, 1);
  LT_TEST(2 * 3 == 6, 1);

  LT_RETURN;
}
```

4. Orchestrator (`main`) Function

The orchestrator:

- Declares and initializes the test runner
- Parses command-line arguments
- Opens the report
- Executes test groups
- Writes category results
- Closes the report

Example:

```
LT_DECLARE_ORCHESTRATOR(main)
{
  LT_INIT_ORCHESTRATOR(main, project, 1);
  LT_PARSE_ARGS(2, "report.txt");
  LT_OPEN_REPORT("Example Report");

  LT_WRITE_RESULT(LT_GROUP(test_math), "Math Tests");

  LT_CLOSE_REPORT(NULL);
  LT_EXIT;
}
```

5. Test Inclusion Control (`-I` / `-In`)

Each `LT_TEST` and `LT_GROUP` macro includes an optional **include parameter** that determines whether it executes based on command-line options.

- `1` — always execute (default)
- `2-9` — execute only when `-In` is provided and `n` is >= the include value
- `I` — execute only when `-I` is provided
- `0` — never execute

This allows selective execution of test subsets.

6. Concurrent Blocks

Concurrent blocks ensure that all enclosed `LT_TEST` calls begin execution together:

```
LT_BEGIN_CONCURRENT(block1);
LT_TEST(expr1, 1);
LT_TEST(expr2, 1);
LT_END_CONCURRENT(block1);
```

Concurrent blocks cannot be nested within a test function.

7. Isolation Modes

LiteTest supports:

Same-Thread Mode (default)

Fastest execution; faults are caught via signal guards.

Thread-Isolated Mode

Each test runs in a separate thread.

Process-Isolated Mode

Each test runs in a separate process. This isolates:

- Aborts
- Memory corruption
- Deadlocks
- Infinite loops
- Sanitizer aborts

Recommended for CI and fault-injection testing.

8. Building the Test Executable

Linux / macOS

```
make run
```

To use gcc:

```
make CC=gcc run
```

Windows (POSIX toolchain required)

Use MSYS2 UCRT64 or Clang64:

```
./build_test_litetest.ps1
.\test_litetest.exe
```

9. Executable Usage

`test_<name> [-I | -In] [PATH]`

- If `PATH` is a file, the report is written to that file.
- If `PATH` is a directory, the default report filename is used.
- If omitted, the report is written to the current directory.

Use `--help` or `-h` to display usage information.

10. Troubleshooting

- Missing POSIX APIs on Windows → use MSYS2 UCRT64 or Clang64
- Report not found → check whether PATH was a file or directory
- Paths with spaces → quote them
- Unexpected behavior → ensure code and docs match the same LiteTest version

11. Further Reading

See:

- **LiteTest Runner Reference** for detailed API semantics.
- **LiteTest Contributor Guide** for development and versioning rules.

Quick Start

LiteTest tests are C/C++ expressions/functions, and the orchestrator controls reporting and execution.

1. To try LiteTest, copy 'litetest_runner.h' and 'litetest_runner.c' to your current directory:

```
cp /path/to/litetest_runner.h .
cp /path/to/litetest_runner.c .
```

2. Create a file named `test_quick.c` in the same directory.
3. Copy and paste the code into `test_quick.c`:

```c
#include "litetest_runner.h"

// A simple test group function.

static LT_DECLARE_GROUP(test_quick)
{
  LT_INIT_GROUP(test_quick, 1);

  int a = 2;
  int b = 2;

  // 4 test assertions.
  LT_TEST(a == b, , 0);         // Pass
  LT_TEST(a + b == 4, ,  0);     // Pass
  LT_TEST(a - b == 1, , 0);     // Fail
  LT_TEST(LT_FAULT(1), , 0);    // Fault

  LT_RETURN;
}

// A simple orchestrator (main) function.

LT_DECLARE_ORCHESTRATOR(main)
{
  LT_INIT_ORCHESTRATOR(main, quick, 1);
  LT_PARSE_ARGS(2, "quick_test_report.txt");
  LT_OPEN_REPORT("Test Quick Report");

  // Single test category.
  LT_WRITE_RESULT(LT_GROUP(test_quick), "Quick tests");

  LT_CLOSE_REPORT("Note: This report is a very simple example of using LiteTest.\n"
                  "Note: Multiple test categories can be added using multiple\n"
                  "      test functions.\n"
                  "Note: Orchestrator (`main`) and test functions can be placed in\n"
                  "      individual modules (.c files).\n"
```

```
                    "Note: Parameters can be set to run tests in parallel, isolate\n"
                    "      a test to a separate thread or process, etc.\n"
                    "Note: The expression for LT_TEST can reference functions to\n"
                    "      provide a more complex test. A non-zero result indicates\n"
                    "      pass and a zero result indicates fail. If a fault occurs\n"
                    "      executing the expression, it is detected and counted in\n"
                    "      the report as a fault.\n"
                    "Note: Larger projects can place files in a more conventional\n"
                    "      layout (e.g., `include/` and `src/`, but this example keeps\n"
                    "      everything in your current directory for simplification.\n"
                    "Note: See README.md for LiteTest for additional API features.\n");
  LT_EXIT;
}
```

4. Build the executable `test_quick` in your current directory:

```
cc -std=c99 -Wall -Wextra -o test_quick test_quick.c litetest_runner.c
```

5. Run it:

```
./test_quick
```

6. View `quick_test_report.txt` in your current directory:

```
less quick_test_report.txt   # Press 'q' to quit
```

Example of the report:

```
LiteTest Report
                              Pass   Fail     Fault
-------------------------------------------------
1. Orchestrator                  4
2. Guard 1 and 2                 6
-------------------------------------------------
                       Total    10
```

See the Core API Macros and other sections for more detail.

See Building the Test Executable for platform-specific notes and options.

Key Features

- Pure C implementation.
- Fault handling with nested guard levels.
- Minimal footprint — a single header and source file define the entire API.
- Straightforward API: simple assertions, categories, and reporting so you can focus on writing tests.
- Comprehensive reporting: pass/fail/fault counts per category and overall totals.
- Test support functions that simplify writing and organizing tests.

API Usage Requirements

LiteTest requires:

- POSIX.1-2001 (IEEE Std 1003.1-2001) compatibility.
- A C99-compliant compiler.

Supported environments:

- Linux, macOS, BSD — fully compatible.
- Windows — requires a POSIX layer such as Cygwin, MSYS2, or WSL.

LiteTest has been exercised in POSIX environments; users should validate behavior in their own systems.

How LiteTest Compares

Framework	Language / Style	Dependencies	Fault Isolation	Strengths	How LiteTest Differs
Unity	C, macro-heavy	None	No	Widely used, simple API	LiteTest adds POSIX signal-based fault isolation and category-level reporting.
cmocka	C, function-based	None	Yes (setjmp)	Mature, feature-rich	LiteTest is smaller, header-driven, and easier to embed in small projects.
Check	C, process-based	POSIX tools	Yes (fork)	Strong isolation, fixtures	LiteTest avoids process spawning and keeps a minimal footprint.
Criterion	C, auto-discovery	libc, POSIX	Yes	Modern, fast, rich output	LiteTest is simpler, portable, and avoids auto-discovery complexity.
LiteTest	C99, macro-driven	None	Yes (POSIX signals)	Minimal, portable, easy to embed	Designed for small C projects needing fault isolation without heavy frameworks.

What LiteTest Does Not Provide

LiteTest focuses on executing tests and reporting results. It does not:

- Generate test source files — users write their own test modules.
- Perform automatic test discovery — tests are invoked explicitly by the orchestrator or from within test functions.
- Provide mocking or stubbing frameworks — users implement their own mocking and stubbing.
- Include built-in setup/teardown systems — users implement their own patterns as needed.
- Handle memory management or leak detection — external tools (e.g., Valgrind) must be used.
- Manage source control or repository structure — LiteTest does not define SCM workflows.
- Integrate with build systems or CI pipelines — users configure these as needed.
- Manage test artifacts such as expected-output ("control") files.
- Define or enforce directory layouts for tests or project structure.
- Promote output files to control files — users handle this workflow manually.
- Track versioning or history of test artifacts.
- Produce rich reporting formats such as JUnit XML or HTML output.
- Provide functionality outside the features explicitly described in this document.

Introduction

LiteTest is a lightweight C/C++ testing framework built around a simple execution model: 1. You write C test expressions (that typically invoke functions) for each test and wrap each of these expressions with the `LT_TEST` macro in a test group function. 2. You wrap each test group function name with the `LT_GROUP` macro in the orchestrator. 3. The orchestrator (`main`) function runs the test group functions and reports results.

The LiteTest Runner API files:

- `include/litetest_runner.h` — public API (typedefs, enums, constants, macros, function declarations, and static inline function definitions).
- `src/litetest_runner.c` — function definitions for the LiteTest framework.

The LiteTest Test API files:

- `include/litetest_test.h` — test support/helper function declarations.
- `src/litetest_test.c` — test support/helper function definitions.

A typical test executable includes:

- An orchestrator (`main`) function that opens the report, invokes test group functions, writes category results, and closes the report.
- Multiple test group functions that execute the tests.
- The LiteTest Runner API files (`litetest_runner.h`, `litetest_runner.c`).
- The LiteTest Test API files (`litetest_test.h`, `litetest_test.c`).
- The headers and source files for the project being tested.

main() |- test_group1() | |- test1 | |- test2 | - ...- test_group2() '- ...

Recommended: Put the orchestrator function in one source file and each test group function in its own source file.

When executed, LiteTest produces a report summarizing tests by category, including:

- Pass/fail/fault counts per category.
- Totals across all categories.
- Notes describing each failure or fault.

Core API

The core API is set of macros used to define the orchestrator and test group functions. These macros fall into 3 types:

Type	Purpose	Naming Pattern
Orchestrator	Define and run the test runner	`LT_DECLARE_*`, `LT_INIT_*`, `LT_*`
Test Group Functions	Define test group functions	`LT_DECLARE_GROUP`, `LT_INIT_GROUP`, `LT_RETURN`
Execution	Execute groups or test expressions	`LT_GROUP`, `LT_TEST`

Macros for the Orchestrator (`main`) Function

- `LT_DECLARE_ORCHESTRATOR(funcname)[;]`
- `LT_INIT_ORCHESTRATOR(funcname, project, maxparallel);`
- `LT_PARSE_ARGS(maxargs, defaultreportfilename);`
- `LT_OPEN_REPORT(title]);`
- test group and test expression macros,
- `LT_WRITE_RESULT(gtm, category);`
- `LT_CLOSE_REPORT(notes]);`
- `LT_EXIT;`

Note: - `funcname` must be main. - `maxargs` must be 2 or greater. The first arg is the executable name. The second optional arg is `PATH`. Additional args are for customization and must be parsed by custom code added to the function. - `gtm` is an `LT_GROUP` or `LT_TEST` macro. - For the first macro, a semicolon is required for a forward declaration; otherwise, omit the semicolon and follow with a definition in `{ }`.

Macros for a Test Group Function

- `LT_DECLARE_GROUP(funcname)[;]`
- `LT_INIT_GROUP(funcname, maxparallel);`
- test expression and test group macros.
- `LT_RETURN;`

Note: - `funcname` must not be main and must be same for the first two macros when defining a test group function. - For the first macro, a semicolon is required for a forward declaration; otherwise, omit the semicolon and follow with a definition in `{ }`.

Macros for Executing a Test Group Function or Test Expression

- `LT_GROUP(funcname, [include], isolation)[;]`
- `LT_TEST(expression, [include], isolation)[;]`

When an `LT_GROUP`or `LT_TEST` macro is passed as the first argument to LT_WRITE_RESULT, omit the trailing semicolon. Otherwise, a semicolon is required.

`funcname`: name of the group function to execute.

`expression`: Test expression which is cast to int. If `expression` returns 0, test failed; otherwise, test passed. If a fault is captured, the test faulted.

`include`:

This parameter controls whether a test group or test expression executes based on the test executable's `-I` or `-In` option (see `-I` and `-In` Option):

- 0 — do not execute.
- 1 — always execute.
- 2 – 9 — execute only when the user specifies an `-In` option and the value is less than or
- `equal to n (a single non-zero digit).`
- I — execute only when the user specifies `-I` without a digit (valid only for LT_TEST), Result is counted as an injected pass/fail/fault as well as the usual a pass/fail/fault count.
- omitted — defaults to `1`.

For `isolation`, see Isolation Modes and Fault Handling.

Special values that can be used in any expression:

- `LT_PASS`: returns 1.
- `LT_FAIL`: returns 0. Note this does force a fail. A fail occurs only if the test expression evaluates to 0. `-LT_FAULT(type)`: causes a fault of the specified type: 1 (`SIGSEGV`). 2 (`SIGABRT`), 3 (`SIGBUS`). For other values of type, LT_FAULT returns 0.

Parallel Execution LiteTest starts up to `maxparallel` test group functions or test expressions concurrently. When one finishes, another begins, until all are complete. `maxparallel` is set by the `LT_INIT_ORCHESTRATOR` and `LT_INIT_GROUP` macros.

Macros for Concurrent Execution

The concurrent block macros ensure that all `LT_TEST` macros inside it start together:

- The `LT_TEST` macros are bracketed with:
  - `LT_BEGIN_CONCURRENT(blockname)`
  - `LT_END_CONCURRENT(blockname)`
- Within a test group function body, a concurrent block cannot be nested inside concurrent block.

Isolation Modes and Fault Handling

LiteTest supports two execution isolation modes that balance speed and fault-isolation. Both modes run tests in a single thread within each process; the difference is whether all test groups and tests run inside one process or each runs in its own process.

Mode	Speed	Isolation	Best Use
0 – Same thread	Fastest	Low	Local dev
1 – Thread	Fast	Medium	Concurrent tests
2 – Process	Slower	Full	CI, fault injection

`isolation`: - 0 same thread (no parallelism) - 1 separate thread - 2 separate process

Defaults: - Tests and test groups not within a concurrent block: 0. - Tests within a concurrent block: 1.

Invalid combinations are rejected.

A fault is handled as follows:

- The fault is recorded
- Execution continues
- The execution completes normally
- The final report includes fault counts and messages

This allows you to test low-level or unsafe code without aborting the entire test run.

1. Single-Process Mode (default)

In this mode, all test groups and tests run sequentially inside a single process and a single thread. This provides the fastest execution and the simplest debugging experience.

LiteTest installs a signal guard that can detect and report certain synchronous faults, including:

- `SIGSEGV` (invalid memory access)
- `SIGBUS` (bus error)
- `SIGFPE` (arithmetic error)
- `SIGILL` (illegal instruction)

These faults can be caught and reported without terminating the test run.

However, some failures cannot be isolated in a single process. If a test triggers one of the following, the entire LiteTest process terminates:

- `SIGABRT` (abort(), assert() failures, malloc corruption).
- `SIGKILL`, `SIGSTOP`.
- External termination signals (`SIGTERM`, `SIGINT`, `SIGHUP`)
- Sanitizer aborts.
- Undefined behavior that escalates to process termination.
- Deadlocks, infinite loops, or resource exhaustion.

Single-process mode is ideal for everyday development and fast feedback, but it does not provide complete fault isolation.

2. Process-Isolated Mode (parallel or serial)

In this mode, each test group function and test expression runs in its own child process. LiteTest monitors each child and reports its result after the process exits.

Because each test group function and test expression runs in a separate process, LiteTest can isolate:

- `SIGABRT` and all abort-based failures.
- Memory corruption that triggers allocator aborts.
- Sanitizer aborts.
- Undefined behavior that terminates the process.
- Deadlocks (child can be timed out and then killed).
- Infinite loops (child can be timed out and then killed).
- Resource exhaustion.
- All synchronous faults (`SIGSEGV`, `SIGBUS`, `SIGILL`, `SIGFPE`).

A failure in one test group cannot affect any other test group or the test runner itself.

Process-isolated mode is recommended for:

- CI environments
- fault-injection testing
- untrusted or experimental code
- tests that may hang, abort, or corrupt memory
- running test groups in parallel

Summary:

Mode	Execution	Isolation Strength	Best For
Single-Process	One process, one thread	Partial (cannot isolate aborts/hangs)	Fast local runs, debugging
Process-Isolated	One process per test group	Full (survives all faults)	CI, fault injection, parallel runs

Both modes use the same `LT_GROUP` and `LT_TEST` macros. The choice of isolation mode affects only how test group functions and test expressions are executed, not how they are written.

LiteTest Runner Customization

Typedefs, enums for exit code and return codes, macros for limits, and functions are provided in the LiteTest Runner API to support customization of the orchestrator and test group functions..

Examples include:

- `lt_executablename`
- `lt_result_t`
- `lt_dirpath`
- `LT_MAX_PATH_LEN`
- `lt_currentlevel`
- `lt_currentresult`
- `lt_maxparallel`
- `lt_blockname`
- `lt_iswritedirpath`

`lt_result_t` carries pass/fail/fault and injected-fault counters for a test or group. A function-level fault is represented by setting the fault count to `SIZE_MAX`.

See the LiteTest Runner Reference for details on each of these.

LiteTest Test API

Typedefs, struct, enums, variables, and Functions are provided to support writing tests.

Examples of Process and runtime helpers include:

- `int lt_execute_command(const char *command_line, int timeout_ms, char *output_buffer, size_t output_buffer_size, int *exit_code)`
- `int lt_wait_for_condition(int (*condition)(void *callback_context), void *callback_context, int timeout_ms, int poll_interval_ms)`

Examples of File and filesystem helpers include:

- `int lt_copy_file(const char *source_path, const char *destination_path)`
- `int lt_make_temp_dir(const char *prefix, char *out_path, size_t out_path_size)`

Examples of Comparison and matching helpers include:

- `int lt_compare_files(FILE *left_file, FILE *right_file)`
- `int lt_compare_file_to_path(FILE *file, const char *path)`
- `int lt_compare_paths(const char *left_path, const char *right_path)`
- `int lt_compare_path_to_file(const char *path, FILE *file)`
- `int lt_match(const char *text, const char *pattern)`

Example of Environment helpers:

- `int lt_with_environment_variable(const char *variable_name, const char *temporary_value, int (*callback)(void *callback_context), void *callback_context)`

See the LiteTest Test Reference for details on each of these.

Headers (.h) and Sources (.c)

Source files containing the orchestrator or test group functions must include `litetest_runner.h` and any required project headers.

Example: Testing lubtype Project

The `tests` directory in the `paulsinclair51/lubtype` repository provides an example orchestrator and test group source (.c) files for the lubtype API. Each source file includes these two headers:

```
#include "lubtype.h"
#include "litetest_runner.h"
```

Example: Self-Testing LiteTest Project

The `tests` directory for this repository provides an example self-test for the LiteTest API and framework.

```
#include "litetest_runner.h"
```

Orchestrator (`main`) Function Template

```
LT_DECLARE_ORCHESTRATOR(main)
{
  LT_INIT_ORCHESTRATOR(main, project, [maxparallel]);
  LT_PARSE_ARGS([maxargs], ["defaultfilename"]);
  LT_OPEN_REPORT(["title"]);

  // Insert group/test/write calls here:
  //   LT_GROUP(funcname, [isolation]);
  //   LT_TEST(expression, [isolation]);
  //   LT_TEST_I(expression, [isolation]);
  //   LT_WRITE_RESULT(LT_GROUP(funcname, [isolation]), "category");
  //   LT_WRITE_RESULT(LT_TEST(expression, [isolation]), "category");
  // Group test/assert calls using:
  //   LT_BEGIN_CONCURRENT(groupname);
  //   LT_END_CONCURRENT(groupname);

  LT_CLOSE_REPORT(["notes"]);

  LT_EXIT;
}
```

`LT_WRITE_RESULT` writes one category and resets its counts. Totals accumulate across the full run.

Optional typedefs, variables, functions, and code may be added to customize or support testing. Added code may use test support functions (see Test Support API) and customization support functions (see Customization Support API).

Recommended: do not intermix code with the tests; such code is handled as if it occurs before the tests.

Forward-declaration:

```
LT_DECLARE_ORCHESTRATOR(main);
```

Test Group Function Template

```
[static] LT_DECLARE_GROUP(funcname)
{
  LT_INIT_GROUP(funcname, [maxparallel]);

  // Insert test/group calls here:
  //   LT_TEST(expression, [isolation]);
  //   LT_TEST_I(expression, [isolation]);
  //   LT_GROUP(funcname, [isolation]);
  // Group test/assert calls using:
  //   LT_BEGIN_CONCURRENT(groupname);
  //   LT_END_CONCURRENT(groupname);

  LT_RETURN;
}
```

Use `static` when the function is only referenced in the same module.

Forward declaration:

```
[static] LT_DECLARE_GROUP(func);
```

Specify `static` if the above definition of the function specifies `static`.

Optional typedefs, variables, functions, and code may be added to customize or support testing. Added code may use test support functions (see Test Support API) and customization suport functions (see Customization Support API).

Recommended: do not intermix code with the tests; such code is handled as if it occurs before the tests.

Example of Using the LiteTest API

The `tests` directory for this repository provides a self-test implementation of the LiteTest API and framework. It includes:

- `test_litetest.c` defines the orchestrator (`main`) with two test categories "Orchestrator" and "Guard".
- `test_orchestrator.c` defines the test_orchestrator function for testing the "Orchestrator" category.
- `test_guard1.c` defines the `test_guard1` function for testing part of the "Guard" category.
- `test_guard2.c` defines the `test_guard2` function for testing the other part of the "Guard" category.

The results of `test_guard1` and `test_guard2` are combined by the orchestrator into a single result for the "Guard 1 and 2" category.

Building the Test Executable

Modify a copy of an existing Makefile that builds a test executable for a project to a Makefile for your project. For example, use the Makefile for testing the lubtype project or the Makefile for self-testing this LiteTest project as a starting point for creating your Makefile in your project root directory.

Linux / macOS This is the primary supported and currently validated environment. Linux is the canonical path for building the test executable, including GitHub Codespaces.

From the repository root:

```
make run
```

The Makefile builds and executes the executable writing the test report in the reports directory with the default name `<project>_test_report.txt`.

The executable can then be executed directly (see Executable Usage).

To build with gcc instead of clang:

```
make CC=gcc run
```

Windows (POSIX Toolchain Required) Use a POSIX-capable toolchain such as MSYS2 UCRT64 or Clang64.

On Windows, use `build_test_litetest.ps1` to build and run the test executable:

```
./build_test_litetest.ps1
.\test_litetest.exe
```

A common setup is MSYS2 UCRT64 with:

```
pacman -S --needed base-devel mingw-w64-ucrt-x86_64-gcc
```

Ensure the MSYS2 `bin` directory is on `PATH` before running PowerShell.

Clang64 is also suitable if `cc`, `clang`, or `gcc` resolves to a POSIX-capable compiler.

Executable Usage

To execute the tests, run the test executable to produce the test report file (an existing writable file is overwritten):

```
test_<testname> [-I|-In] [PATH]
```

By default, if `PATH` is not specified, LiteTest writes the report to the current working directory using the default report filename configured by your test setup.

PATH

You may override the output location using the PATH argument:

- `PATH` can point to either a report file or a directory. Quote it only when it contains spaces (for example, `"my reports/"`).
- If `PATH` is a file path (existing or new), LiteTest writes the report to that file.
- If `PATH` is a directory path, LiteTest writes the report in that directory using the default report filename configured by your test setup.

`-I` and `-In` Option

The `-I` and `-In` options control which tests execute based on the `include` parameter of the `LT_GROUP` and `LT_TEST` macros. `n` is a single non-zero digit. Only one of these forms may be specified.

- Use `-In` to enable `LT_GROUP` and `LT_TEST` macros that have an `include` argument that is a non-zero digit less than or equal to `n`.
- Use `-I` to enable `LT_TEST` macros that have an `include` argument that is `I`. This can be used to exercise the LiteTest framework and verify report formatting (typically, these `LT_TEST` macros have a test expression that is coded to cause a failures or a fault.
- If neither option is provided, all `LT_GROUP` and `LT_TEST` macros with an `include` argument that is `1` – `9` execute by default.

See Macros for Executing a Test Group Function or Test Expression for the `include` parameter and its interaction with the `-I` and `-In` options.

`--help` and `-h` Help Options

You can display usage information at any time with the `--help` or `-h` option. For example:

```
./test_litetest --help
```

This (or using -h instead of –help) prints a summary of all command-line options and usage details.

Common Mistakes

- If `PATH` contains spaces, quote it (for example, `"my reports/report.txt"`).
- Existing report files may be overwritten; use unique paths if you need history.
- On Windows, use a POSIX-capable toolchain (for example, MSYS2 UCRT64).
- Keep macro examples in your project aligned with the version of `litetest_runner.h` in use.

Troubleshooting

- Build fails with missing POSIX APIs on Windows: verify you are using a POSIX-capable toolchain (for example, MSYS2 UCRT64) and that its `bin` directory is on `PATH`.
- Report file not found where expected: confirm the current working directory and check whether `PATH` was passed as a file path or directory path.
- Output path with spaces fails: quote `PATH` (for example, `"my reports/report.txt"`).
- Unexpected behavior after macro updates: ensure code and docs match the same LiteTest version (`LT_VERSION` in `litetest_runner.h` and `LT_VERSION_C` in `litetest_runner.c`).

Example Test Report

```
LiteTest Report
                              Pass   Fail     Fault
-------------------------------------------------
1. Orchestrator                  4
2. Guard 1 and 2                 6
-------------------------------------------------
                       Total    10
```

Example Test Report for -i Option

```
LiteTest Report (-i)

                             Pass   Fail     Fault
-------------------------------------------------
1. Orchestrator                 4      1         1
2. Guard 1 and 2                6      2         1
-------------------------------------------------
                       Total    10      3         2


```{=latex}
\clearpage
\ltsetrunninghead{Further Reading}
\global\ltaftertoctrue
```

Further Reading

- include/README.md
- src/README.md
- tests/README.md
- reports/README.md

** * @section Overview Overview @note In the following, testing the LiteTest itself is used as an example of * using the LiteTest framework and API with test modules test_guards_1.c, * test_guards_2.c, and test_orchestrator.c, and test orchestrator * test_litetest.c in the repository tests directory. */

/** * @subsection KeyPoints Key Points 1. A test executable is built from: - An orchestrator (main) function and optional test functions * organized into one or more modules, - litetest_runner.h, and litetest_runner.c, unistd.h - Modules and include files from the feature/project/API under test. *
* 2. The executable produces a report grouped by category, including * pass/fail/fault counts per category and totals across all categories. * Fail and fault messages are appended to the report. 3. The orchestrator and test functions may reside in one module * or multiple modules. Recommended: put the orchestrator function * in one module and each test function in its own module. 4. The test framework requires unistd.h for POSIX fork and signal capabilities. */

/** * @subsection OrchestratorMacros Orchestrator Function Macros Macros for use in the orchestrator (main) function: - LT_DECLARE_ORCHESTRATOR(funcname)[;] * - LT_INIT_ORCHESTRATOR(funcname, testsuitename, [maxparallel]); * - LT_PARSE_ARGS([maxargs], [“defaultreportfilename”]); * - LT_OPEN_REPORT([“reporttitle”]); * - test and assert macros * - LT_WRITE_RESULT([t], “categoryname”); * - LT_CLOSE_REPORT([“notes”]); * - LT_EXIT; @note funcname must be main. * @note maxargs must be 2 or greater. The first arg is the executable name. * The second optional arg is PATH. Additional args are for customization * and must be parsed by custom code added to the function. * @note t is
```

a test or assert macro. * @note For the first macro, a semicolon is required for a forward declaration; * otherwise, it is omitted if is it followed by a definition in { }. */

/** * @subsection TestFunctionMacros Test Function Macros Macros for use in a test function: - LT_DECLARE_TEST(funcname)[;] * - LT_INIT_TEST(funcname, [maxparallel]); * - test and assert macros * - LT_RETURN; @note funcname must not be main and must be same for the first two macros when * defining a test function. * @note For the first macro, a semicolon is required for a forward declaration; * otherwise, it is omitted if it is followed by a definition in {}. */

/** * @subsection TestAndAssertMacros Test and Assert Macros - LT_TEST(funcname, [isolation])[;] * - LT_ASSERT(expression, [isolation])[;] * - LT_INJECT_ASSERT(expression, [isolation])[;] The semicolon is omitted if used as an argument to the LT_WRITE_RESULT macro; * otherwise it is required. LT_INJECT_ASSERT is the same as LT_ASSERT except: - Only executes if injection is enabled (see @ref iOption `-i` Option. * - Result is counted as an injected pass/fail/fault. Special values that can be used in any expression: - LT_PASS: returns 1. * - LT_FAIL: returns 0. * - LT_FAULT(type): causes a fault of the specified type: *. 1 (`SIGSEGV`). 2 (`SIGABRT`), 3 (`SIGBUS`).* For other values of type, LT_FAULT returns 0. */

/** * @subsubsection FaultHandling Fault Handling LiteTest provides multi-level signal guards to safely capture faults such as * `SIGSEGV` `SIGABRT`, and `SIGBUS`. When a fault occurs: - The fault is recorded * - Execution continues * - The test suite completes normally * - The final report includes fault counts and messages This allows you to test low-level or unsafe code without aborting the entire * test run. */

/** * @subsubsection ParallelExecution Parallel Execution

Parallel execution of the test/assert macros is enabled/disabled by the `maxparallel` parameter for the LT_INIT_ORCHESTRATOR and LT_INIT_TEST macros. Up to `maxparallel` test/assert macros are started and when one finishes another is started. */

/** * @subsubsection ParallelGroupMacros Parallel Group Macros

A group ensures that all test/assert macros inside it start together:

- Groups are bracketed with:
  - `LT_BEGIN_GROUP(groupname)`
  - `LT_END_GROUP(groupname)`
- Within a test function, a group cannot be nested inside another group.

*/

/** * @subsubsection Isolation Levels * `isolation`: - 0 same thread (no parallelism) - 1 separate thread - 2 separate process

Defaults: - Non-grouped macros: 0. - Grouped macros: 1.

- Invalid combinations are rejected. */

/** old * @subsubsection ParallelExecution Parallel Execution Parallel execution of the test/assert macros is enabled/disabled by the * maxparallel parameter for the LT_INIT_ORCHESTRATOR and LT_INIT_TEST * macros. Up to maxparallel test/assert macros are started and when one * finishes another is started. @subsubsection

ParallelGroupExecution Parallel Group Execution Test/assert macros can run in parallel as a group (that is, they * are not started until they can all start without exceeding maxparallel). * A group is bracketed using the followug macros: - LT_BEGIN_GROUP(groupname, [isolation]) * - LT_END_GROUP(groupname); @note A group cannot be nested in a group within a test function. @subsubsection TextIsolation Test Isolation For non-grouped test/assert macro, the isolation parameter * indicates whether the macro runs in the same thread as the calling * function (no parallelism), a separate thread, or a separate process: Isolation: 0 (none), 1 (thread), 2 (process) @note For a test/assert macro not in a group, the default is 0 (none). @note For a test/assert macro in a group, isolation must be 1 (thread) * or 2 (process) with a default of 1 (thread). */

/** * @subsection Miscellaneous * * Miscellaneous functions, macros, typedefs, and variables. * Examples include: - lt_executablename * - lt_result_t * - lt_dirpath * - LT_MAX_PATH_LEN * - lt_currentlevel * - lt_currentresult * - lt_maxparallel * - lt_isisolated * - lt_groupname * - lt_iswritedirpath */

/** * @section HeaderUsage Header Usage * * In the test orchestrator source file (e.g., test_litetest.c), * include the following: @code #include “litetest_runner.h” * @endcode Use the orchestrator macros, variables and functions in the test * orchestrator logic plus the LT_TEST macro to execute test functions. * * In the test* modules (e.g., test_guards_1.c, test_guards_2.c, and * test_orchestrator.c): @code #undef LT_ORCHESTRATOR #include “litetest_runner.h” * @endcode Use the LT_TEST, LT_ASSERT_FAIL, and LT_ASSERT_FAULT macros in * the test function to execute tests. @note Other veriations are possible in the test orchestrator and test functions. */

/** * @section BuildingTestExecutable Building a Test Executable * * - Linux/macOS: make * * Defaults to use Makefile in the current directory. * * - Windows PowerShell: `.\build_test_lubtype.ps1` */

/** * @section NameConventions Naming Conventions LiteTest - Repository name (case-jnsensitive. litetest_runner.h and litetest_runner.c - filenames. * * Public API: 1. lt_* - functions, typedefs, and variables. * 2. LT_* - macros, constants, and enum values. * * Internal and private to the LiteTest framework: 1. litetest_* - functions, typedefs, and variables. * 2. LITETEST_* - Internal macros, constants, and enum values. These conventions are designed to provide a clean public API, strong * namespace isolation, and predictable behavior when LiteTest is embedded * into a larger C/C++ project. @example Public API Names 1. Utility macros: LT_TOK_PASTE, LT_TOK_STR, LT_RESULT, LT_TOTAL * LT_STATIC_ASSERT * 2. Version macros: LT_VERSION, LT_VERSION_EQ, LT_VERSION_AT_LEAST * 4. Utility functions: lt_current_level. @example Public typedef Name lt_result_t, lt_state_t @example Private Variable Names litetest_result_internal, litetest_total_internal */

/** @section FaultGuarding Fault Guarding The test framework uses multi-level fault guarding to handle * faults (i.e., segmentation fault, bus error, or abort) during * test execution. * * A test/assert macro wraps a guard around its argument, enabling * detection of a fault not handled at a lower level by the * argument rather than aborting. This allows counting pass, * fail, and fault without aborting due to a fault. * * A fault detected by LT_TEST represents a fault * in the use of the testing framework or in the testing framework * itself, and not in the feature being tested. Such a fault is * expected to be rare but guarding

avoids a fault terminating * execution. */